

Knapsack – Industry Resource Allocation

Maximizing Cloud Revenue with 0/1 Knapsack Dynamic Programming

Student: ADITHYA R

Register Number: 192311369

Domain: Design and Analysis of Algorithms

Technique Used: 0/1 Knapsack — Dynamic Programming

1. Problem Analysis

A cloud service provider operates a physical/virtual server with a fixed memory capacity of 100 GB. Multiple independent client applications are requesting deployment on this server. Each application A_i requires a fixed amount of memory w_i (in GB) and, if deployed, generates a fixed daily revenue v_i (in revenue units) for the provider. Because server memory is a hard physical limit, and because a cloud tenant is either running on the server or not — partial deployment of an application does not exist — the provider must choose a subset of applications to host such that total memory used does not exceed 100 GB, while total revenue is maximized.

This is a direct real-world instance of the 0/1 Knapsack Problem: the server's memory capacity plays the role of the knapsack capacity W , each client application plays the role of an item with weight w_i (memory) and value v_i (revenue), and the binary decision 'host this application or not' plays the role of the 0/1 inclusion variable $x_i \in \{0, 1\}$. The provider cannot deploy 60% of an application to save memory — an application either receives its full memory allocation or is not deployed at all — which is precisely the indivisibility constraint that distinguishes 0/1 Knapsack from the fractional variant.

1.1 Sample Client Application Data

The following five client applications are being evaluated for deployment on a server with 100 GB of available memory:

Application	Memory Required (GB)	Revenue (units/day)	Revenue/GB
App A	20	500	25.0
App B	30	700	23.3
App C	10	300	30.0
App D	40	900	22.5
App E	25	600	24.0

Server capacity $W = 100$ GB. Objective: choose a subset $S \subseteq \{A, B, C, D, E\}$ maximizing $\sum \text{revenue}(i)$ for $i \in S$, subject to $\sum \text{memory}(i)$ for $i \in S \leq 100$.

2. Applying the 0/1 Knapsack Model

- Items $\rightarrow$ Client applications requesting deployment.
- Weight of item $i \rightarrow$ Memory requirement of application i (GB).
- Value of item $i \rightarrow$ Daily revenue generated by application i if deployed.
- Knapsack capacity $W \rightarrow$ Total available server memory (100 GB).
- Binary decision variable $x_i \in \{0,1\} \rightarrow$ Whether application i is deployed (1) or rejected/deferred (0), with no partial deployment allowed.

The optimization objective is therefore: maximize $\sum v_i \cdot x_i$ subject to $\sum w_i \cdot x_i \leq W$ and $x_i \in \{0,1\}$ — which is the canonical 0/1 Knapsack formulation applied directly to server-memory provisioning.

3. Dynamic Programming Solution

3.1 Recurrence Relation

```

dp[i][w] = maximum revenue achievable using the first i applications
           within w GB of memory

dp[0][w] = 0                                for all w
dp[i][w] = dp[i-1][w]                        if memory[i] > w
dp[i][w] = max( dp[i-1][w],
                revenue[i] + dp[i-1][w - memory[i]] ) otherwise

Answer = dp[n][W]   where n = number of applications, W = 100

```

3.2 DP Table (selected memory columns, in GB)

Apps \ w	0	20	40	60	80	100
∅	0	0	0	0	0	0
+A (20GB,500)	0	500	500	500	500	500
+B (30GB,700)	0	500	700	1200	1200	1200
+C (10GB,300)	0	500	1000	1500	1500	1500
+D (40GB,900)	0	500	1000	1500	1900	2400
+E (25GB,600)	0	500	1000	1500	1900	2400

Note: $dp[4][100]$ already reaches 2400 after considering only Apps A–D — this is the value produced by combining A+B+C+D (weight exactly 100 GB). Adding App E in the final row leaves $dp[5][100]$ unchanged at 2400, since no combination that includes E can match or beat the revenue of A+B+C+D filling the full 100 GB. Reading the bottom-right cell, $dp[5][100] = 2400$, gives the optimal total revenue.

3.3 Backtracking the Optimal Deployment Plan

Tracing $dp[5][100] = 2400$ back through the table: comparing $dp[5][100]=2400$ to $dp[4][100]=2400$ shows no change, so App E was NOT included. Comparing $dp[4][100]=2400$ to $dp[3][100]=1500$ shows a change, so App D WAS included — move to $w = 100 - 40 = 60$. Comparing $dp[3][60]=1500$ to $dp[2][60]=1200$ shows a change, so App C WAS included — move to $w = 60 - 10 = 50$. Comparing $dp[2][50]=1200$ to $dp[1][50]=500$ shows a change, so App B WAS included — move to $w = 50 - 30 = 20$. Finally $dp[1][20]=500$ versus $dp[0][20]=0$ confirms App A WAS included. This trace yields the final optimal deployment set:

Application Deployed	Memory Used (GB)	Revenue (units/day)	Status
App A	20	500	Deployed
App B	30	700	Deployed
App C	10	300	Deployed
App D	40	900	Deployed
App E	25 (unused)	0	Rejected
Total	100 / 100	2400	—

Optimal Plan: deploy Apps A, B, C, and D, using the full 100 GB of server memory and generating 2400 revenue units/day. App E is rejected — not because its revenue-per-GB ratio is poor (24.0, the second highest of all five apps), but because no combination that includes it can reach 100 GB as efficiently as A+B+C+D does. This is the key insight DP captures that a purely ratio-based (greedy) selection would miss.

4. Justification of the Dynamic Programming Approach

A naive provisioning heuristic might sort applications by revenue-per-GB and greedily deploy the highest-ratio applications first (the correct approach for divisible resources, but not for indivisible ones). Running that heuristic on this exact dataset — order C(30.0) → A(25.0) → E(24.0) → B(23.3) → D(22.5) — deploys C, A, E, and B (memory 10+20+25+30 = 85 GB, revenue 300+500+600+700 = 2100), then cannot fit D (40 GB) into the remaining 15 GB and leaves 15 GB of paid-for capacity idle. This greedy plan earns only 2100 revenue units/day — 12.5% less than the DP-optimal 2400, and it also wastes 15 GB of provisioned memory that generates no revenue at all.

Dynamic programming is justified here because it does not commit irrevocably to any single application based on its ratio; instead, it evaluates every relevant combination of include/exclude decisions through the recurrence, allowing it to discover that trading App E for the combination of A+B+C+D fills memory more completely and yields strictly higher total revenue. In an industry setting where memory is a hard, non-negotiable constraint and revenue directly impacts the provider's bottom line, this 12.5% revenue gap is exactly the kind of loss that DP-based allocation eliminates and greedy allocation would silently leave on the table every single day.

5. Overlapping Subproblems and Optimal Substructure

5.1 Optimal Substructure

The problem exhibits optimal substructure because the optimal deployment plan for capacity w using the first i applications is built directly from optimal solutions to smaller subproblems: either the optimal plan for $i-1$ applications at the same capacity w (App i excluded), or the optimal plan for $i-1$ applications at reduced capacity $w - \text{memory}[i]$, plus $\text{revenue}[i]$ (App i included). Because the best overall plan is guaranteed to be

assembled from best sub-plans — never from a suboptimal sub-plan — the recurrence in Section 3.1 is valid and sufficient to compute the true global optimum.

5.2 Overlapping Subproblems

The same subproblem $dp[i][w]$ is required repeatedly across different decision paths in the table. For example, $dp[2][60]$ (the best revenue using only Apps A and B within 60 GB) is needed both when deciding whether to include App C at capacity 60 (via $dp[3][60] = \max(dp[2][60], \dots)$) and again when deciding whether to include App D at capacity 60+ combinations that reduce back down to 60 GB after subtracting App D's 40 GB. Without memoization, a naive recursive implementation would recompute $dp[2][60]$ independently every time it is needed — and this duplication compounds exponentially with each additional application. By storing each $dp[i][w]$ result exactly once in the table (memoization/tabulation), the DP solution guarantees every distinct subproblem is solved a single time, which is what reduces the algorithm from exponential-time brute force (2^n possible subsets, here $2^5 = 32$) down to the polynomial-in- $(n \times W)$ table-filling procedure shown in Section 3.2.

6. Suitable Tools and Platforms for Implementation

- Programming languages: Python (with NumPy for vectorized DP-table operations) for rapid prototyping and integration with cloud SDKs; C++/Java for latency-sensitive production allocation engines where the DP table must be recomputed frequently.
- Optimization libraries: Google OR-Tools or PuLP/COIN-OR for formulating the same problem as an Integer Linear Program (ILP) — useful when additional real-world constraints (SLA priorities, multi-resource limits such as CPU + memory + bandwidth together) need to be layered on top of the basic knapsack model.
- Cloud orchestration platforms: The allocation decision integrates naturally with Kubernetes (via custom schedulers or admission controllers), AWS Auto Scaling / AWS Batch, or Azure Resource Manager, where the DP-computed deployment plan becomes the input to the platform's actual placement/scheduling API.
- Caching / memoization infrastructure: Redis or an in-memory cache can store previously computed $dp[i][w]$ subproblem results when allocation is recomputed frequently (e.g., every few minutes as new client requests arrive), avoiding full table recomputation from scratch.
- Monitoring and reporting: Prometheus + Grafana dashboards to track realized revenue vs. DP-predicted optimal revenue over time, helping validate that the model's assumptions (fixed memory/revenue per app) still hold in production.
- Data pipeline: A simple relational store (PostgreSQL/MySQL) or a spreadsheet-based tool for smaller deployments to maintain the live table of client applications, their memory requirements, and contracted/estimated revenue values that feed into the DP computation.

7. Performance Analysis — Time and Space Complexity

Implementation	Time Complexity	Space Complexity
DP — full 2D table	$O(n \times W)$	$O(n \times W)$

DP — rolling 1D array	$O(n \times W)$	$O(W)$
Brute-force (all subsets)	$O(2^n)$	$O(n)$

For this instance, $n = 5$ applications and $W = 100$ GB, so the DP table requires only $6 \times 101 = 606$ cells — computed in a fraction of a millisecond, and vastly cheaper than the $2^5 = 32$ -subset brute-force enumeration would suggest is necessary at this small scale (though brute-force is still feasible here, it stops being feasible once n grows past roughly 25–30 applications). The rolling-array optimization (keeping only the current and previous row) reduces space from $O(n \times W)$ to $O(W)$ and is recommended for production use, since only the final optimal value and a separately-tracked selection list are typically needed, not the full historical table.

The pseudo-polynomial nature of this complexity is the critical scaling consideration in an industry deployment: time and space depend on the numeric magnitude of W (memory capacity in GB), not just on the number of applications n . A single high-memory server (e.g., $W = 2,048$ GB at 1 GB granularity) with $n = 500$ candidate applications would require a DP table of roughly $500 \times 2,048 \approx 1$ million cells — still comfortably fast (well under a second) on modern hardware — but if capacity were tracked at megabyte granularity instead of gigabyte granularity, W could balloon a thousandfold and make the full DP table impractical, motivating either coarser memory-granularity buckets (e.g., allocate in fixed 1 GB blocks, as most cloud providers already do) or a switch to an ILP solver with cutting-plane/branch-and-bound techniques for very large-scale instances.

8. Interpreting the Results in an Industry Context

- Revenue optimization: The DP solution identifies that hosting Apps A, B, C, and D fully utilizes the 100 GB server and yields the maximum possible 2400 revenue units/day — a concrete, auditable allocation decision the provider can act on immediately.
- Capacity utilization: The optimal plan uses exactly 100 GB (0% idle capacity), compared to the greedy alternative's 85 GB usage (15% idle capacity) while earning 12.5% less revenue — a double inefficiency that DP-based scheduling avoids.
- Rejected-tenant handling: App E, though individually attractive (2nd-highest revenue-per-GB), is correctly deferred; in practice this application could be queued for the next available server, offered a smaller memory tier, or flagged for a follow-up negotiation on pricing/SLA — the DP model gives the provider a defensible, quantitative reason for the decision rather than an ad hoc one.
- Repeatability at scale: Because DP recomputation is cheap at realistic problem sizes (Section 7), the provider can re-run this optimization every time a new client request arrives, a contract changes, or a server is added/removed from the pool, keeping the allocation continuously revenue-optimal rather than fixed at initial deployment time.
- Risk consideration: Because 0/1 Knapsack treats revenue and memory as fixed, known constants, the provider should periodically revalidate these inputs (e.g., revenue may be usage-based rather than flat, memory needs may grow) and re-run the DP allocation rather than treating a single computed plan as permanent.

9. Conclusion

The cloud memory-allocation problem maps precisely onto 0/1 Knapsack: applications are items, memory requirements are weights, revenue values are item values, and total server memory is the capacity constraint. Dynamic programming is the correct and necessary technique here because a simple revenue-per-GB greedy heuristic — while optimal for divisible resources — provably underperforms on this indivisible-allocation problem, leaving both revenue (12.5% lower) and capacity (15% idle) on the table. By exploiting optimal substructure and eliminating the recomputation of overlapping subproblems through tabulation, the DP approach computes the guaranteed-optimal deployment plan (Apps A, B, C, D; revenue 2400/day; 100% capacity utilization) in time and space that remain practical for realistic industry-scale server and tenant counts, provided memory is tracked at a sensible granularity.